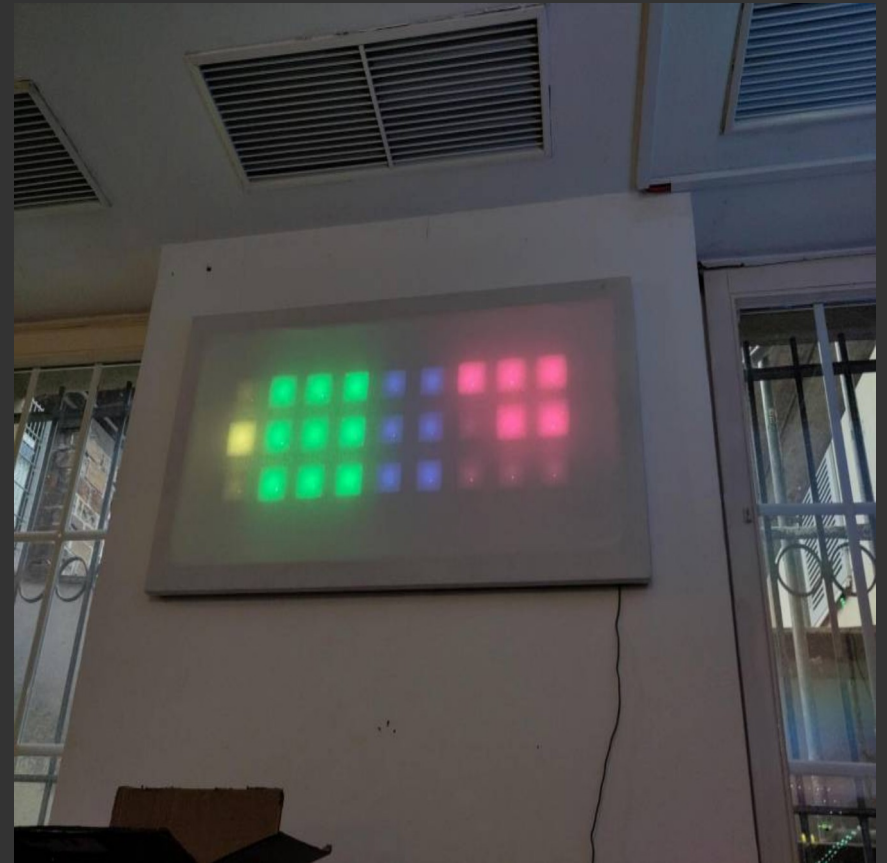
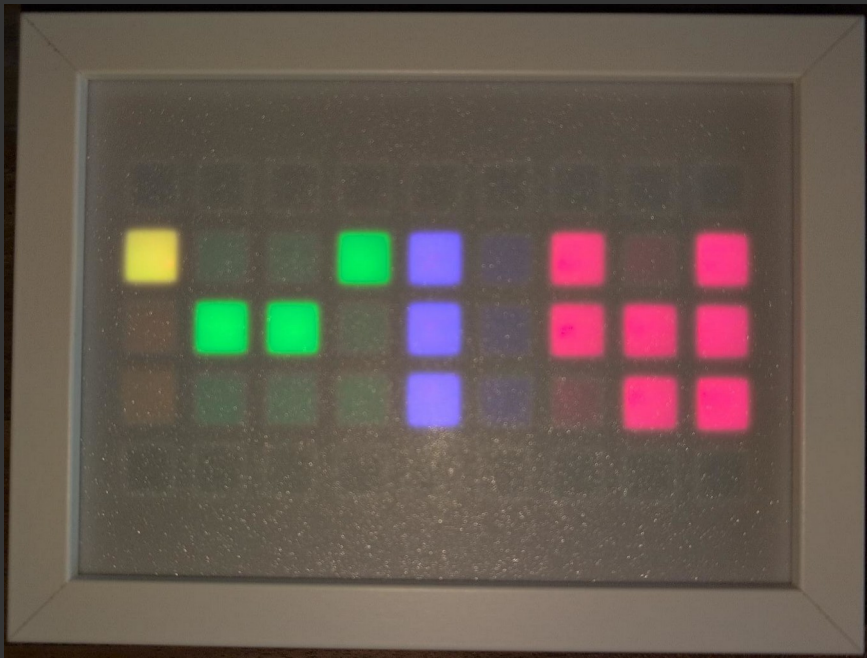


**S.p.a.c.e. U.h.r.**

# SpaceUhr

Von

Magic  
Mystery  
Muesli



# Magic Mystery Muesli

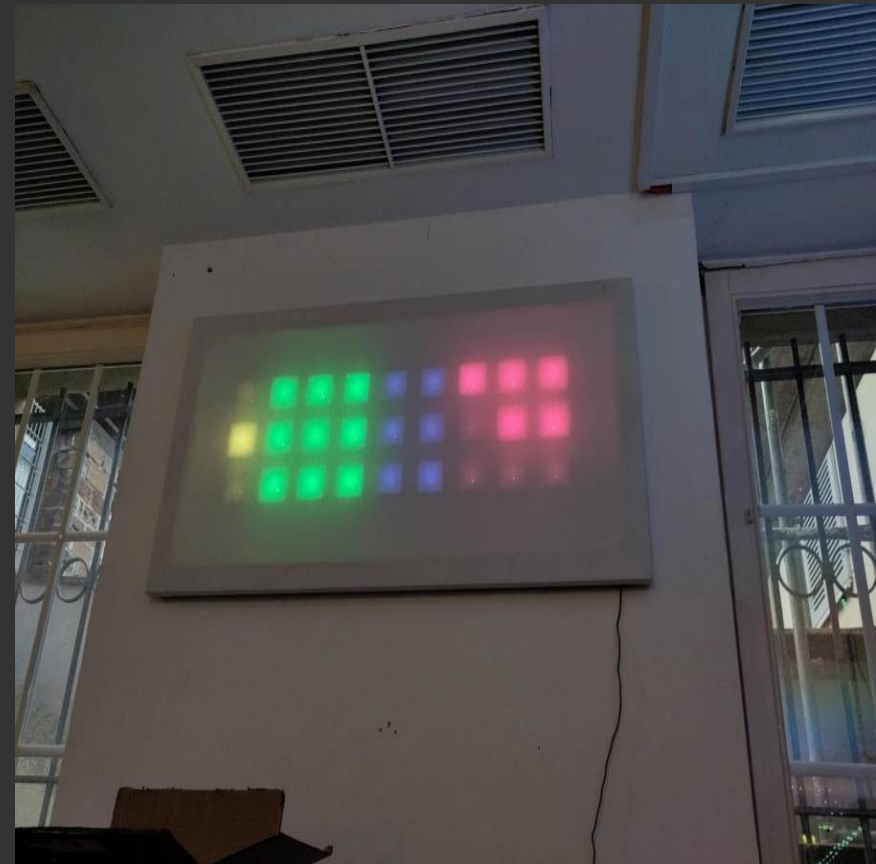
- HAXKO
- technikbegeisteter Elektroingenieur
- PLC, KNX, Softwareentwicklung
- [https://woof.tech/@dr\\_muesli](https://woof.tech/@dr_muesli)

# butzel

- HAXKO & LUG-MYK
- FIAE & IT-Berater
- Python, C, Java, VFP
- GNU/Linux (Gentoo)
- Pentester (RedTeamer)

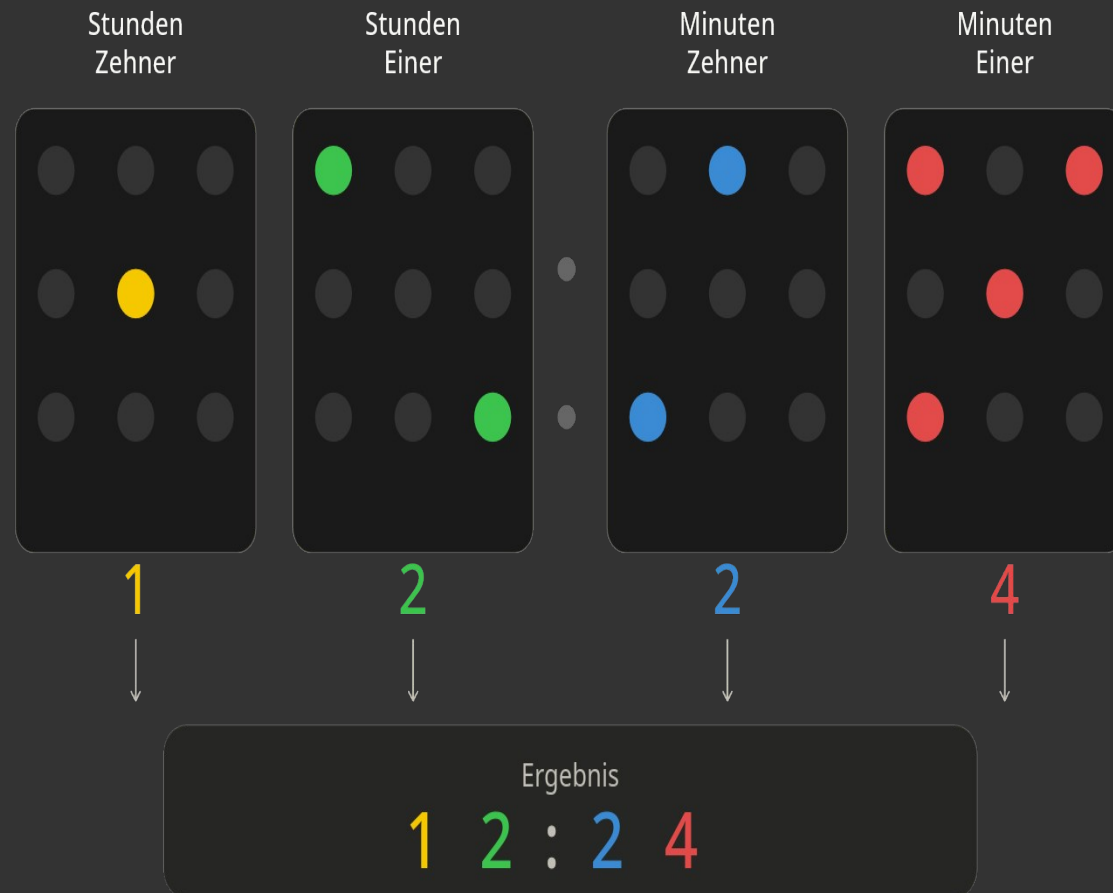
# SpaceUhr

- Idee nach der Tix-Uhr
- alternative Darstellungsarten



# SpaceUhr: Funktionsweise

- keine BinärUhr
- Leuchtpunkte zählen

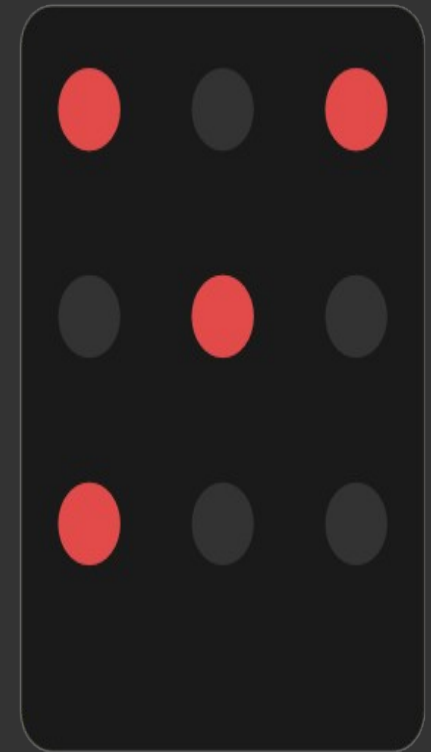
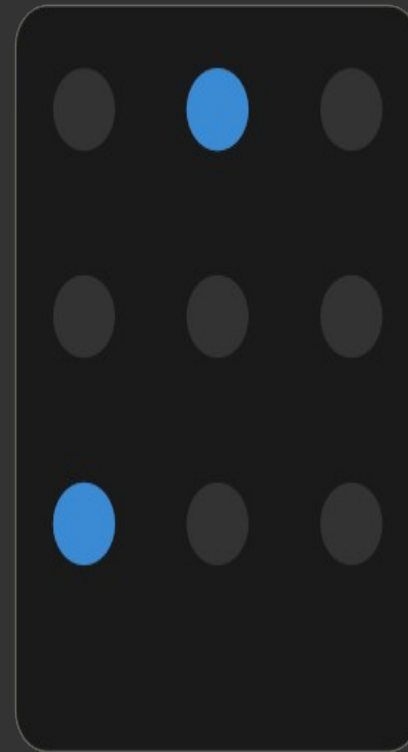
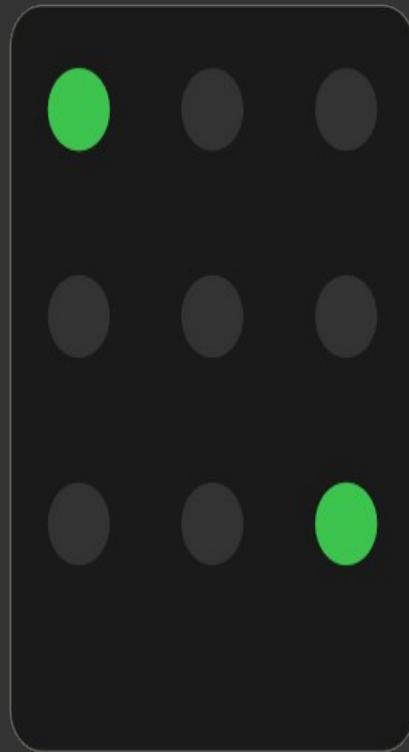
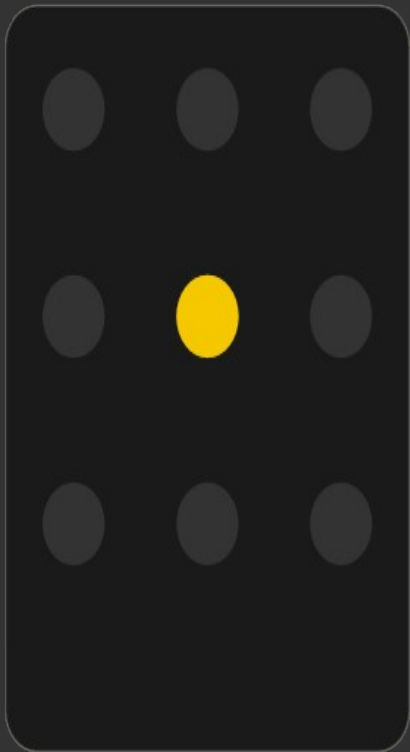


Stunden  
Zehner

Stunden  
Einer

Minuten  
Zehner

Minuten  
Einer



1



2



2



4

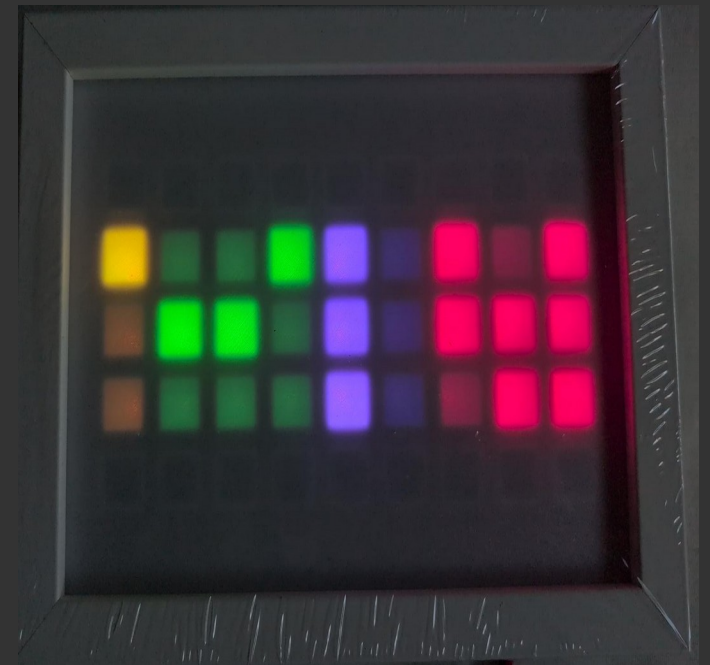


Ergebnis

1 2 : 2 4

# SpaceUhr@EH23

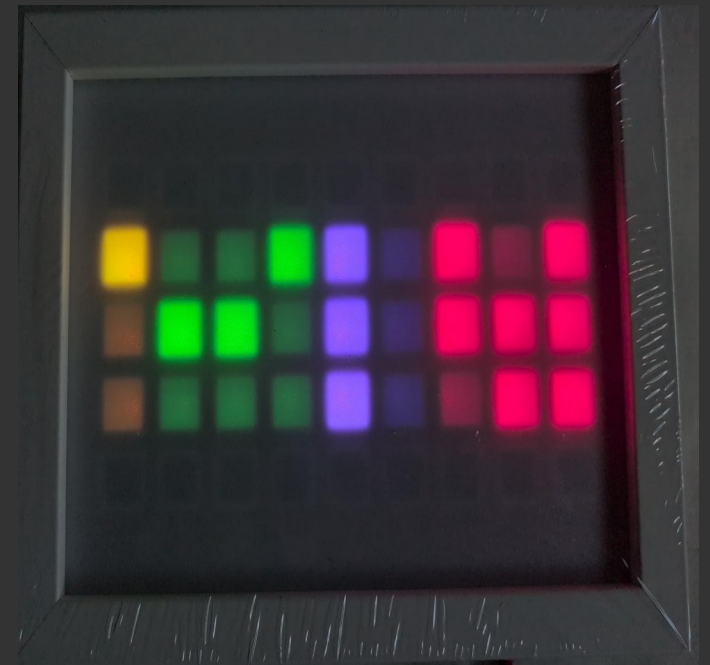
- verkleinert
- Bilderrahmen „Rödalm“
- 5-zeilig (45 NeoPixel)
- kein Taster
- ca. 15-20€





# SpaceUhr: BoM

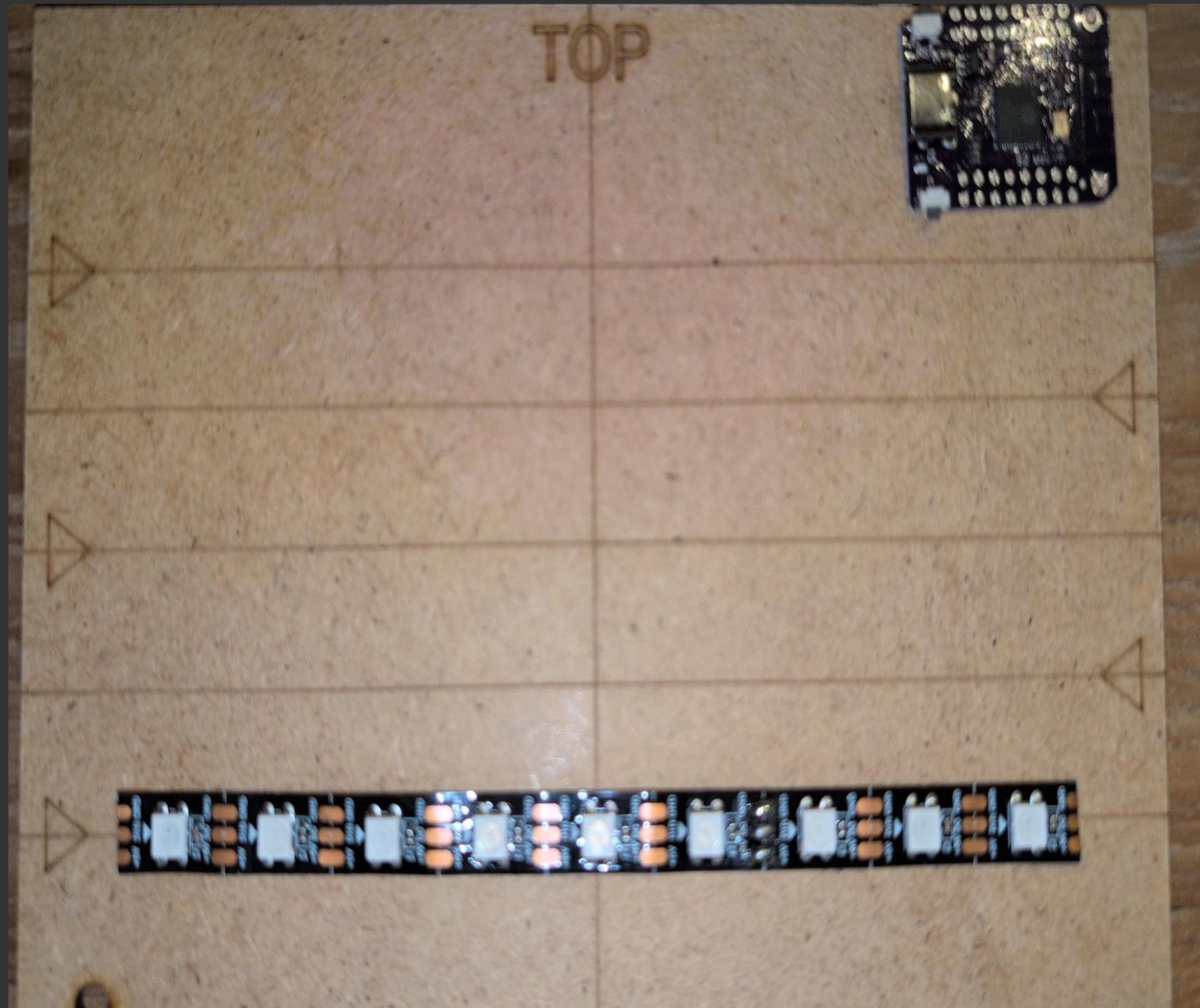
- Bilderrahmen „Rödal“
- 5-Streifen NeoPixel
- ESP32
- 3D-Druck
- Kabel



# SpaceUhr: Aufbau I

- NeoPixel  
(selbstklebend)
- Bilderrahmen

Pfeilrichtung  
beachten!



# SpaceUhr: Aufbau I

- NeoPixel  
(selbstklebend)
- Bilderrahmen

Pfeilrichtung  
beachten!





# SpaceUhr: Aufbau II

- NeoPixel  
(selbstklebend)
- Bilderrahmen
- Kabel

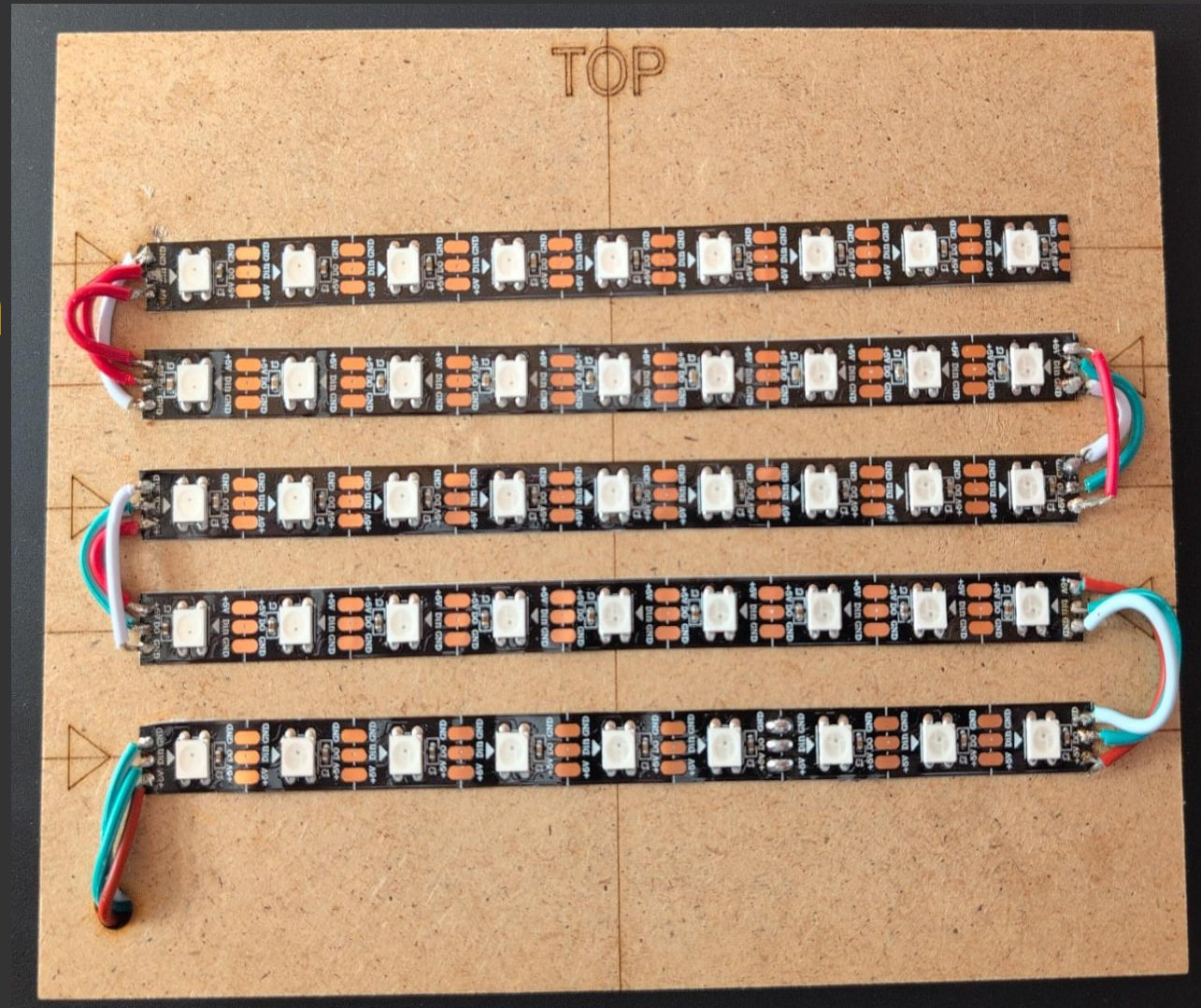
# Beschriftung beachten!



# SpaceUhr: Aufbau II

- NeoPixel (selbstklebend)
- Bilderrahmen
- Kabel

Beschriftung beachten!

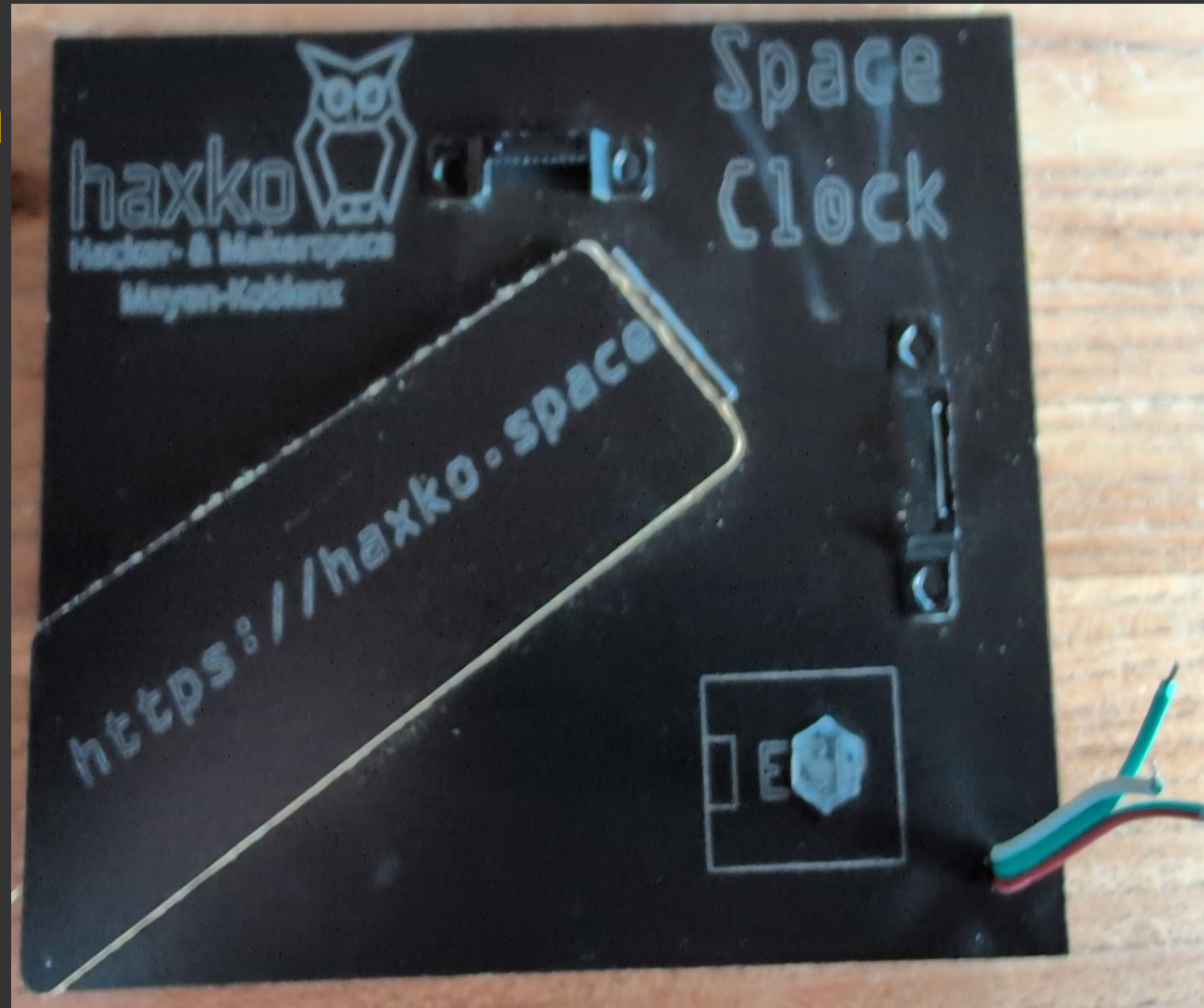




# SpaceUhr: Aufbau III

- Bilderrahmen
- ESP32

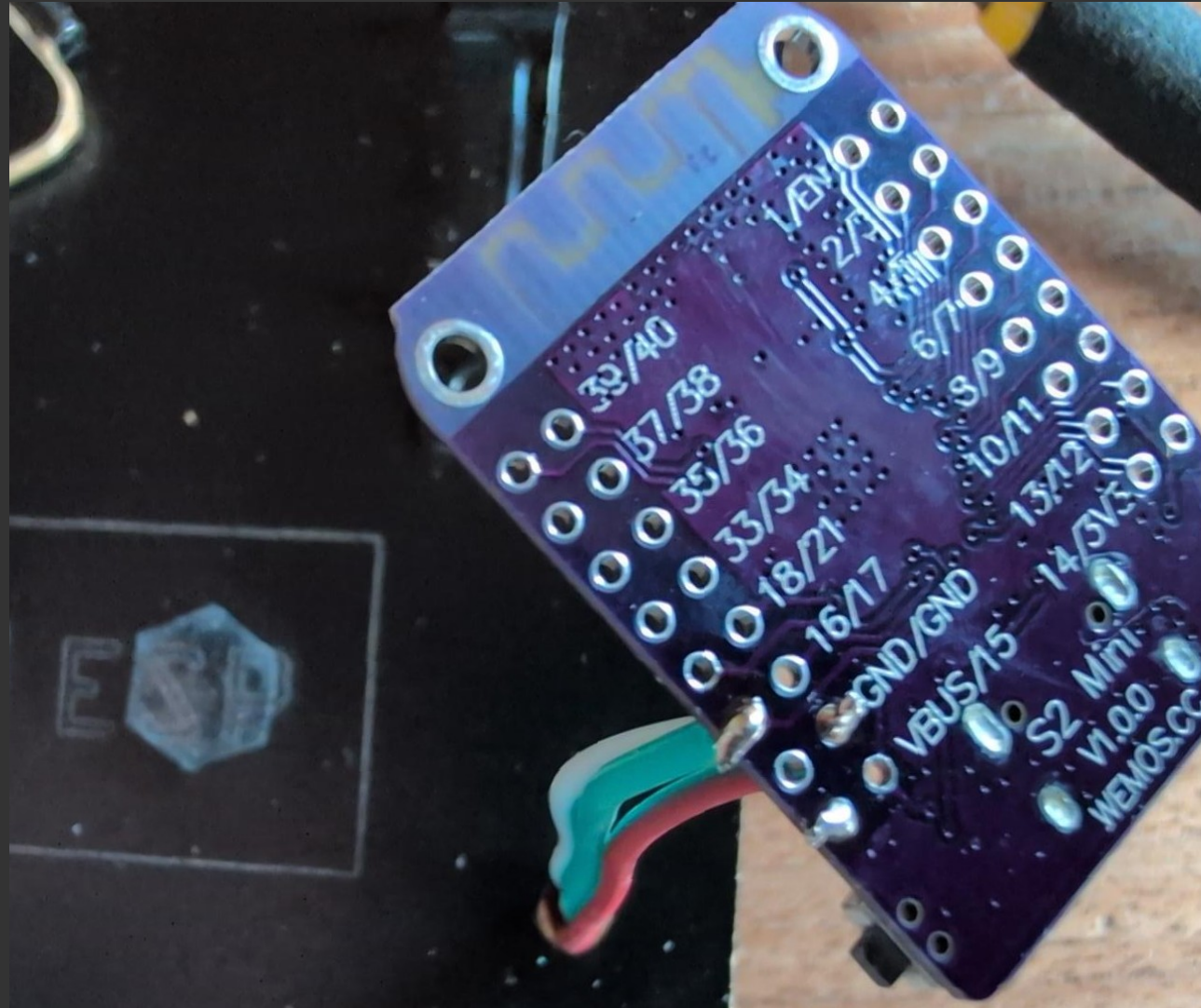
Pin 16: DATA  
Pin GND: GND  
Pin VBUS: 5V



# SpaceUhr: Aufbau III

- Bilderrahmen
- ESP32

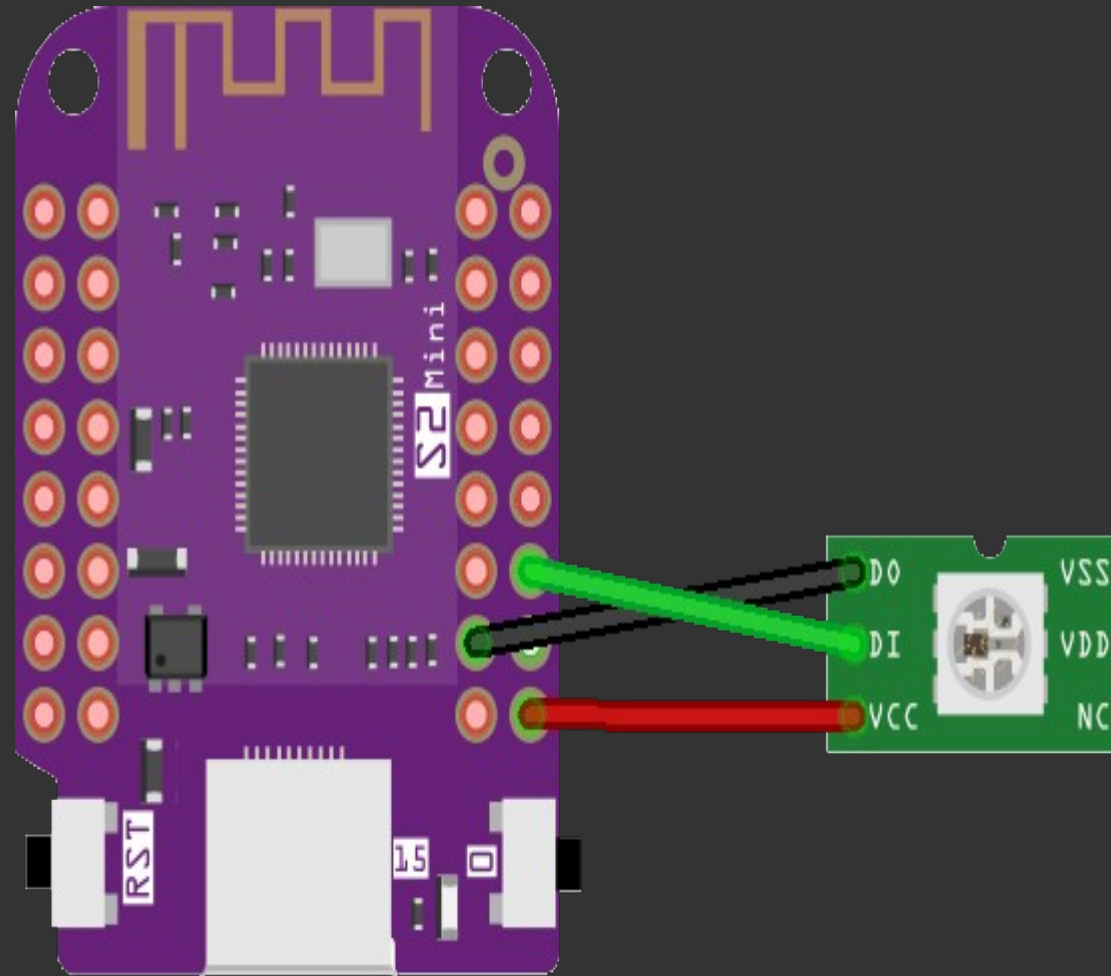
Pin 16: DATA  
Pin GND: GND  
Pin VBUS: 5V



# SpaceUhr: Aufbau IIII

- Bilderrahmen
- ESP32

Pin 16: DATA  
Pin GND: GND  
Pin VBUS: 5V





# SpaceUhr: Flashen

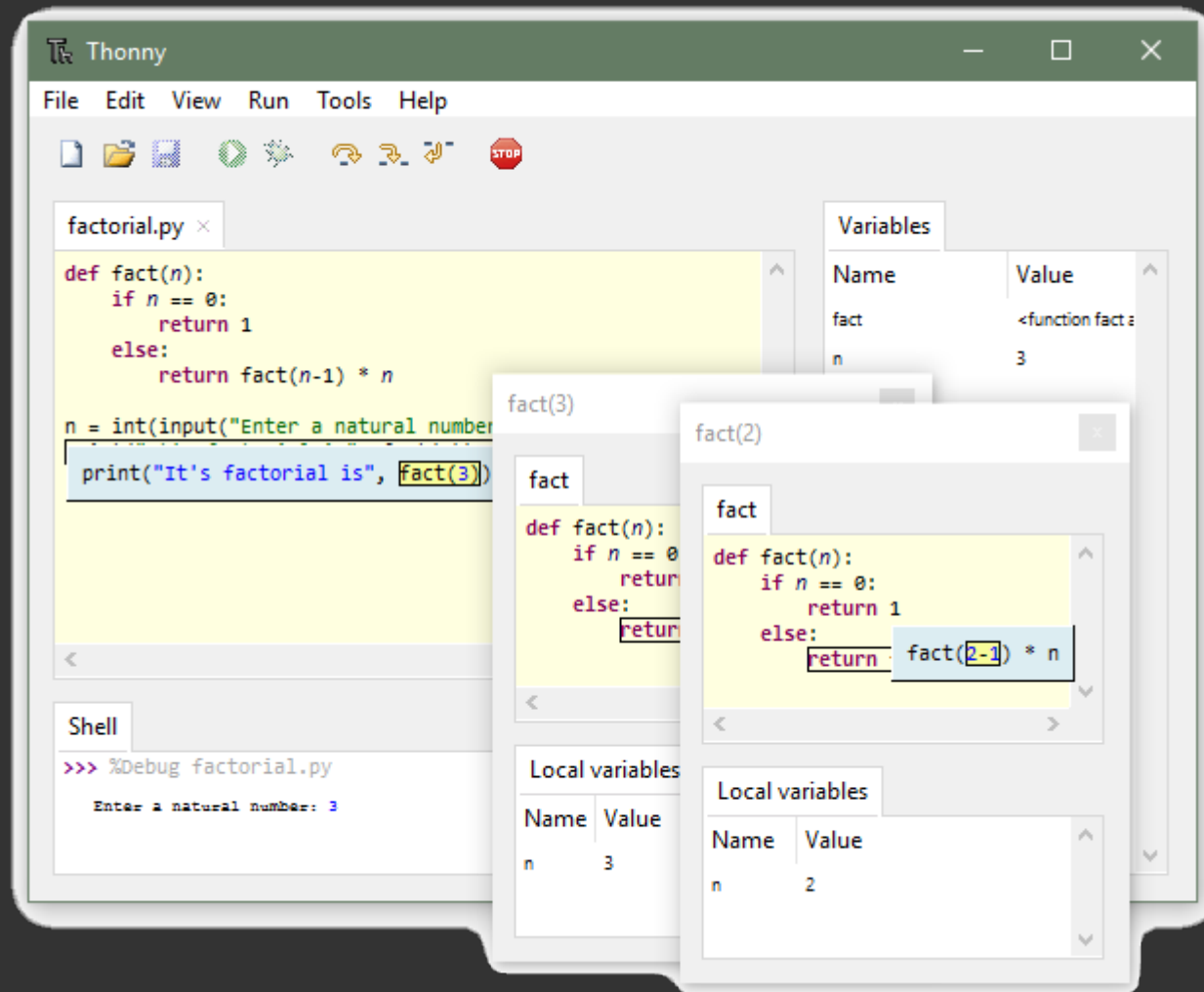
- ESP32
- `esptool` bzw. `esptool.py`

```
# boot-Taster gedrückt halten & USB verbinden  
# boot-Taster lösen
```

```
esptool --no-stub --port /dev/ttyUSB0 erase_flash
```

```
esptool --no-stub --port /dev/ttyACM0 write_flash 0x1000 µpython.bin
```

# SpaceUhr: thonny



# SpaceUhr: Programmieren

- thonny
  - Menüpunkt: run
  - Reiter: interpreter
  - DropDown: MicroPython (ESP32)
  - [strg]+[F2]

# SpaceUhr: main.py

- thonny
  - Menüpunkt: File/Open
  - Dialog: This computer
  - Datei: main.py



# SpaceUhr: main.py

- thonny
  - Menüpunkt: File/Save As...
  - Dialog: MicroPython device
  - Datei: main.py

# SpaceUhr: wlan\_data.py

- thonny
  - Menüpunkt: File/Open
  - Dialog: MicroPython device
  - Datei: wlan\_data.py

# SpaceUhr: wlan\_data.py (anpassen)

- wlan\_data.py

```
WLAN_NETZE = [  
    ("ssid", "<password>"),  
    ("freifunk-myk.de", ""),  
    ("example", "easterhegg2026"),  
]
```

# SpaceUhr: wlan\_data.py

- thonny
  - Menüpunkt: File/Save As...
  - Dialog: MicroPython device
  - Datei: wlan\_data.py

# SpaceUhr: MicroPython

- Erlaeuterung (aus mp-workshop: 11)
- Besonderheiten (aus mp-workshop: 27)
- Flashen (aus mp-workshop: 15)
- Verbinden (aus mp-workshop: 31)
-



# SpaceUhr: `import abc.py`

- Font-Library: `abc.py`  
`import abc`
- `setPixel(x, y, rgb, pin)`  
`setPixel(2,4, (7,14,21), 16)`
- `write(str, rgb, pin)`  
`write("Hi", (7,14,21), 16)`
- `marquee(str, rgb, delay, loops, pin)`  
`marquee("haxko.space", (7,7,7), 0.2, 5, 16)`



# ::> MicroPython

:: Python 3

:- Mikrocontroller

:- Embedded Systems

:: Systemanforderungen

:- 128 KB Flash

:- 8 KB RAM

:: Damien P. George

:: 2013 Kickstarter

:: 2014 Release

:: 2016 ESP8266

:: 2016 PyCom (ESP32)

:: 2017 CircuitPython

:: Plattformen

:- unix

:- pyboard

:- stm32f4

:- **esp8266**

:- esp32

:- Calliope Mini 2.0

:- RP2040 (ARM Cortex M0+)

:- ...

# :> Software: esptool.py

:: Flashen von ESP-Chips

:- Installation über PIP:  
`pip install esptool`

:- MAC auslesen:  
`esptool.py -p /dev/ttyUSB0 -b 115200 read_mac`

:- Speicher leeren:  
`esptool.py -p /dev/ttyUSB0 -b 115200 erase_flash`

:- MicroPython flashen  
`esptool.py -p /dev/ttyUSB0 -b 115200  
write_flash -fm dio --flash_size=8m 0 μpython.bin`

# ::> Software: Terminal

:: screen

:- screen /dev/ttyUSB0 115200

:: PicoCom

:- picocom /dev/ttyUSB0 -b115200

:: SoCat

:- socat TCP4-LISTEN:8266,fork,reuseaddr  
/dev/ttyUSB0,raw,crnl,b115200

# ::> Software: butzel.net

:: escaper.py [<http://files.butzel.net/escaper.tgz>]

- :- transferiert Dateien
- :- reduziert Code
- :- kombiniert Imports

:: mptools (fs.py) [<http://files.butzel.net/opensource/>]

- :- Dateisystembefehle  
rm, cp, mv, cat, ls
- :- Systembefehle  
reset, turbo, gc, meminfo, df
- :- Wlan & Co  
ifconfig, wlanap, wscan, i2cscan



# :> MicroPython & Python

:: Unterschiede

:: Grenzen

:: Besonderheiten

- :- boot.py

- :- main.py

- :- WebREPL

# :> Flashen von MicroPython

:: Port testen:

```
dmesg | grep -i uart
```

```
[ 3.1337 ] usb 1-1.2: ch341-uart converter now attached to ttyUSB0
```

:: testen:

```
esptool.py -p /dev/ttyUSB0 -b 115200  
read_mac
```

:: Speicher leeren:

```
esptool.py -p /dev/ttyUSB0 -b 115200  
erase_flash
```

:: Flashen von MicroPython

```
esptool.py -p /dev/ttyUSB0 -b 460800  
write_flash -fm dio -fs=detect 0 firmware.bin
```

# :> Verbindungsaufnahme

:: Connect

```
screen /dev/ttyUSB0 115200
```

:- Hallo Welt

```
>>> print("Hallo Welt!")
```

:- mehr Hallo Welt

```
>>> def say_hello():  
...     print("Hallo")
```

```
>>> say_hello()
```

:: mptools installieren

```
./escaper.py -rc fs.py -m main.py -d /dev/ttyUSB0
```

# :> Wlan: Starten

```
>>> help( )
```

```
>>> import network
```

```
>>> wlan = network.WLAN(network.STA_IF)
```

```
>>> wlan.active(True)
```

```
>>> wlan.scan( )
```

# :> Wlan: Scannen

```
>>> import network
>>> wlan = network.WLAN(network.STA_IF)
>>> wlan.active(True)
>>> for netz in wlan.scan():
...     print( str( netz[0], 'ASCII' ) )
```

*scandone*

*Honigmond*

# :> Wlan: Verbinden

```
>>> import network
>>> wlan = network.WLAN(network.STA_IF)
>>> wlan.active(True)
>>> wlan.connect(b'Honigmond', b'llabdrEWelle')
ip:10.1.1.XX,mask:255.255.255.0,gw:10.1.1.1
```

# :> Wlan: Konfiguration

```
>>> import network
>>> wlan = network.WLAN(network.STA_IF)
>>> wlan.active(True)
>>> wlan.connect(b'Honigmond', b'llabdrEWelle')
ip:10.1.1.XX,mask:255.255.255.0,gw:10.1.1.1

>>> wlan.ifconfig( )
('IP','Netzmaske','Gateway','DNS')
```



# :> WebREPL: setup

```
>>> import webrepl_setup
```

*WebREPL daemon auto-start status: disabled*

*Would you like to (E)nable or (D)isable it running on boot?*

*(Empty line to quit)*

```
> e
```

*To enable WebREPL, you must set password for it*

*New password (4-9 chars): password*

*Confirm password: password*

*Changes will be activated after reboot*

*Would you like to reboot now? (y/n) y*

# **:> WebREPL: start**

```
>>> webrepl.start( )
```

```
WebREPL daemon started on ws://192.168.4.1:8266
```

```
WebREPL daemon started on ws://10.1.1.xx:8266
```

**:: WebREPL**

**:- <http://micropython.org/webrepl>**

```
WebREPL verbindet direkt zum ESP im lokalen Netzwerk (und nicht über micropython.org ;-)
```

# ::> Credits

:: SpaceUhr

:- Haxko:

:- <https://haxko.space>

:: Bild von Thonny

:- Wikipedia

:- [https://upload.wikimedia.org/wikipedia/commons/d/d4/Thonny2\\_0\\_screenshot\\_windows\\_10.png](https://upload.wikimedia.org/wikipedia/commons/d/d4/Thonny2_0_screenshot_windows_10.png) (CC BY-SA 4.0)

:: Platinen Bilder

:- Fritzing

:- [Adafruit.com](https://adafruit.com)